

Limits of Computation

Assignment 1 (Deadline 5.03.2026, 4pm)

You need to submit this coursework electronically at the correct E-submission point. You must submit a zipped directory that contains TWO files with the exact names as described below:

- a *pdf* file `questions.pdf` containing the answers to Questions 1–4. Please make sure that *all* answers for Questions 1–4 are in *that single document*. In case you use Word, please ensure that you convert your file into a pdf document before submission¹.
- a runnable WHILE source file `STEPrep.while` that contains the answers to Question 5. Note that the WHILE-program for Question 5 is only allowed to call macros that are published on our Canvas site and must run in `hwhile` or `whilers`. For the marking process of your assignment it is essential that you follow the above rules.
- Please use a standard zip program to zip the directory containing the files mentioned above. Please do not use other compression programs or formats as this might give you 0 marks as the markers may be unable to unzip. Please do *not* submit .rar files.
- Please do not write your name anywhere, but it is advisable to *include your candidate number as comment in each submitted file*.
- Please make sure you *check after submission* that you actually have submitted the correct zipped directory of files at the correct submission point.

YOU MUST WORK ON THE ASSIGNMENT ON YOUR OWN! The standard rules for collusion and plagiarism apply and any cases discovered will be reported and investigated. Use of generative AI is not permitted.

¹In Word, this works usually by using the printing menu and then saving as pdf instead of sending to a printer.

Questions

1. A *list of lists of numbers* is a list such that all its elements are lists of numbers. Consider the following examples:

- $[[3, 5], [], [1, 1, 1]]$ is a list of lists of numbers;
- $[[[3, 5]], [], [1]]$ is not a list of lists of numbers due to the first element;
- $[2, [1]]$ is not a list of lists of numbers due to the first element not being a list of numbers, but note that the tree that encodes this list also encodes $[[0, 0], [1]]$ which *is* a list of list of numbers.

Note that an empty list can always be considered a list of numbers and a list of lists of numbers.

The (semantics of) language WHILE uses the binary tree data type $\mathbb{D}$. Consider the following elements in $\mathbb{D}$:

- (a) $\langle \langle \text{nil}.\langle \text{nil}.\text{nil} \rangle \rangle.\langle \langle \text{nil}.\langle \text{nil}.\langle \text{nil}.\text{nil} \rangle \rangle \rangle.\langle \langle \text{nil}.\text{nil} \rangle.\text{nil} \rangle \rangle \rangle$
 (b) $\langle \langle \langle \text{nil}.\text{nil} \rangle.\langle \text{nil}.\langle \text{nil}.\text{nil} \rangle \rangle \rangle.\langle \langle \langle \text{nil}.\text{nil} \rangle.\text{nil} \rangle.\langle \langle \langle \text{nil}.\langle \text{nil}.\text{nil} \rangle \rangle.\langle \langle \text{nil}.\text{nil} \rangle.\text{nil} \rangle \rangle.\text{nil} \rangle \rangle \rangle$
 (c) $\langle \langle \langle \text{nil}.\text{nil} \rangle.\langle \text{nil}.\text{nil} \rangle \rangle.\langle \langle \langle \text{nil}.\text{nil} \rangle.\text{nil} \rangle.\text{nil} \rangle.\langle \langle \text{nil}.\langle \text{nil}.\text{nil} \rangle.\text{nil} \rangle \rangle.\text{nil} \rangle \rangle \rangle$

Answer for EACH of the given trees (a)–(c) above BOTH of the following questions:

- i Does it encode a *list of numbers*? If it does which is the corresponding list of numbers?
- ii Does it encode a *list of lists of numbers*? If it does which is the corresponding list of lists of numbers?

Use our encoding of datatypes in $\mathbb{D}$ as explained in the lectures. [18 marks]

2. Given the following WHILE-program as data d , write a source WHILE-program p such that $\ulcorner p \urcorner = d$.

```

[0, [
  [:=, 1, [cons, [quote, nil], [cons, [quote, nil], [var, 0]]]],
  [while, [var, 1], [
    [if, [tl, [tl, [var, 1]]], [
      [:=, 2, [cons, [var, 1], [var, 2]]]
    ], []],
    [:=, 1, [tl, [var, 1]]]
  ]]
], 2]

```

Make sure your program has correct core WHILE-syntax. [12 marks]

3. Let `prog` be the WHILE-program in Figure 1.

```

prog read X {
  Z := cons nil nil;
  while X {
    A := X;
    B := Z;
    P := nil;
    while A {
      C := B;
      D := P;
      while C {
        D := cons nil D;
        C := tl C
      };
      P := D;
      A := tl A
    };
    Z := P;
    X := tl X
  }
} write Z

```

Figure 1: WHILE-program `prog`

Explain what program `prog` in Figure 1 returns as output for a given input $d \in \mathbb{D}$. In other words, state what $\llbracket \text{prog} \rrbracket^{\text{WHILE}}(d)$ is for any $d \in \mathbb{D}$.

Do not narrate how the program executes step by step, but provide/describe the *behaviour* (semantics) it implements. Your description should explain

the result for *any* input $d \in \mathbb{D}$, and thus should refer to the input tree.

[20 marks]

4. Consider the language REPEAT which has the same datatype $\mathbb{D}$ and program structure as the WHILE-language, and whose syntax differs from WHILE only in the following aspect: commands do *not* contain `while` but instead a `repeat` command the syntax of which is outlined in Figure 2 together with the remaining syntax.

$\langle expression \rangle$	$:=$	$\langle variable \rangle$	(variable expression)
		<code>nil</code>	(atom nil)
		<code>cons</code> $\langle expression \rangle$ $\langle expression \rangle$	(construct tree)
		<code>hd</code> $\langle expression \rangle$	(left subtree)
		<code>tl</code> $\langle expression \rangle$	(right subtree)
$\langle block \rangle$	$::=$	$\{ \langle statement-list \rangle \}$	(block of commands)
		$\{ \}$	(empty block)
$\langle statement-list \rangle$	$::=$	$\langle command \rangle$	(single command list)
		$\langle command \rangle ; \langle statement-list \rangle$	(list of commands)
$\langle elseblock \rangle$	$:=$	<code>else</code> $\langle block \rangle$	(else-case)
$\langle command \rangle$	$::=$	$\langle variable \rangle := \langle expression \rangle$	(assignment)
		<code>repeat</code> $\langle expression \rangle * \langle block \rangle$	(repeat statement)
		<code>if</code> $\langle expression \rangle \langle block \rangle$	(if-then)
		<code>if</code> $\langle expression \rangle \langle block \rangle \langle elseblock \rangle$	(if-then-else)
$\langle program \rangle$	$::=$	$\langle name \rangle$ <code>read</code> $\langle variable \rangle$	
		$\langle block \rangle$	
		<code>write</code> $\langle variable \rangle$	

Figure 2: BNF grammar for REPEAT

Variables and expressions in REPEAT are exactly as in WHILE, and therefore the stores used in the semantics of REPEAT are like the ones used in WHILE. In REPEAT, the semantics of expression `nil`, `cons`, `hd`, `tl`, and of assignment, sequential composition and conditional are exactly like in WHILE.

The semantics of the new command `repeat E * B` is as follows: assuming that E evaluates to $\ulcorner n \urcorner$, executing `repeat E * B` is equivalent to executing B exactly n times (B is therefore not executed at all if $n = 0$). If, however, E does *not* evaluate to the encoding of a natural number, the *program* (in which the `repeat` command is executed) terminates immediately, ignoring any commands that follow the `repeat` command.

For example, for the REPEAT program `sample`:

```
sample read X {
  repeat cons nil nil * { X := tl X }
}
write X
```

we have that $\llbracket \text{sample} \rrbracket^{\text{REPEAT}}(\ulcorner 3 \urcorner) = \ulcorner 2 \urcorner$.

(a) Consider the REPEAT program `rp`:

```
rp read X {
  Y := cons nil nil;
  repeat X * { Y := cons nil cons nil Y };
  Y := cons nil Y
}
write Y
```

- i. What is $\llbracket \text{rp} \rrbracket^{\text{REPEAT}}(\ulcorner 0 \urcorner)$? [4 marks]
- ii. What is $\llbracket \text{rp} \rrbracket^{\text{REPEAT}}(\ulcorner 2 \urcorner)$? [4 marks]
- iii. What is $\llbracket \text{rp} \rrbracket^{\text{REPEAT}}(\ulcorner [1, 2] \urcorner)$? [4 marks]

You do not have to explain your answers.

(b) Can we program a self-interpreter for language REPEAT like we have done for WHILE? Explain your answer *briefly* in at most four sentences. [12 marks]

5. In order to implement the interpreter for REPEAT you are given the file `repeat.while` on Canvas. You are asked to provide the step macro, so please *only submit* the step macro `STEPrep.while`. This step macro must also work fine for REPEAT-programs that don't use the `repeat` statement.

We first need to agree on a programs-as-data encoding for REPEAT. We use the exact same encoding as for WHILE but need to add the encoding of the new repeat statement:

$$\ulcorner \text{repeat } E * B \urcorner = \ulcorner [\ulcorner 4 \urcorner, \ulcorner E \urcorner, \ulcorner B \urcorner] \urcorner$$

Since there is no `@repeat` atom available in `hwhile`/`whilers`, we use the number 4 instead (to encode atom `@repeat`). And if you need other auxiliary atoms, choose numbers to encode them, but make sure your choice won't clash with any of the atoms used already in the step macro (Tip: avoid prime numbers).

Submit the answer to this question in a separate file `STEPrep.while` that is runnable in `hwhile` or `whilers`. You may use any `WHILE`-extension features available in those interpreters. You may use macro calls to any program *published on the Canvas site* including answers to questions on Exercise Sheet 3 and 4 (once published). In the latter case, please don't include them in your submission. You *must not* call any other macros, i.e. programs. Test your submission using `hwhile` or `whilers` before you submit. Programs that don't run will get zero or very few marks. Add comments to your code where appropriate to help the marker understand your thought process.

One must be able to successfully run the program `repeat.while` as interpreter for `REPEAT` using your `STEPrep` macro. This is how we will test your answer and you should test yours in this way before submitting. Also make sure you test that your code does not lead to non-termination. This often happens when one does not clear the command stack properly. [26 marks]